

Practical 4:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>
#include <time.h>

#define MESSAGE_SIZE 1024
#define NUM_MESSAGES 100000

int main() {
    int pipefd[2];
    pid_t pid;

    if (pipe(pipefd) == -1) {
        perror("pipe failed");
        exit(EXIT_FAILURE);
    }

    pid = fork();
    if (pid < 0) {
        perror("fork failed");
        exit(EXIT_FAILURE);
    }

    if (pid == 0) {
        // --- CHILD PROCESS (Consumer) ---
        close(pipefd[1]); // Close unused write end

        char buffer[MESSAGE_SIZE];
        long total_bytes_received = 0;
        ssize_t bytes_read;

        while ((bytes_read = read(pipefd[0], buffer, MESSAGE_SIZE)) > 0) {
            total_bytes_received += bytes_read;
        }
    }

    } else {
        // --- PARENT PROCESS (Producer) ---
        close(pipefd[0]); // Close unused read end

        char data[MESSAGE_SIZE];
        memset(data, 'A', MESSAGE_SIZE); // Fill dummy payload

        struct timespec start, end;
        clock_gettime(CLOCK_MONOTONIC, &start);

        for (int i = 0; i < NUM_MESSAGES; i++) {
            if (write(pipefd[1], data, MESSAGE_SIZE) == -1) {
                perror("write failed");
                break;
            }
        }

        close(pipefd[1]); // Close write end to send EOF to consumer
        wait(NULL); // Wait for child to finish

        clock_gettime(CLOCK_MONOTONIC, &end);

        // Calculate total time taken in seconds
        double time_taken = (end.tv_sec - start.tv_sec) +
            (end.tv_nsec - start.tv_nsec) / 1e9;

        double total_megabytes = (double)(NUM_MESSAGES * MESSAGE_SIZE) / (1024 * 1024);
        double throughput = total_megabytes / time_taken;

        printf("\n--- Performance Metrics ---\n");
        printf("Data Transferred : %.2f MB (%d messages of %d bytes)\n", total_megabytes, NUM_MESSAGES, MESSAGE_SIZE);
        printf("Time Taken : %.6f seconds\n", time_taken);
        printf("Throughput : %.2f MB/sec\n", throughput);
    }

    return 0;
}
```

```
bhanu@DESKTOP-DE32HQ8:~$ nano producer_consumer.c
bhanu@DESKTOP-DE32HQ8:~$ gcc -O2 producer_consumer.c -o producer_consumer
./producer_consumer
[Consumer] Finished receiving. Total bytes: 102400000 bytes

--- Performance Metrics ---
Data Transferred : 97.66 MB (100000 messages of 1024 bytes)
Time Taken       : 0.049505 seconds
Throughput       : 1972.65 MB/sec
bhanu@DESKTOP-DE32HQ8:~$ |
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pipefd[2];
    pid_t pid1, pid2;

    // 1. Create pipe
    if (pipe(pipefd) == -1) {
        perror("pipe failed");
        exit(EXIT_FAILURE);
    }

    // 2. Fork first child for "ls -l"
    pid1 = fork();
    if (pid1 < 0) {
        perror("fork 1 failed");
        exit(EXIT_FAILURE);
    }

    if (pid1 == 0) {
        // --- CHILD 1: ls -l ---
        // Redirect standard output to pipe's write end
        dup2(pipefd[1], STDOUT_FILENO);

        // Close redundant pipe descriptors
        close(pipefd[0]);
        close(pipefd[1]);

        // Execute "ls -l"
        char *args[] = {"ls", "-l", NULL};
        execvp(args[0], args);

        perror("execvp ls failed");
        exit(EXIT_FAILURE);
    }
}
```

```

// 3. Fork second child for "grep .c"
pid2 = fork();
if (pid2 < 0) {
    perror("fork 2 failed");
    exit(EXIT_FAILURE);
}

if (pid2 == 0) {
    // --- CHILD 2: grep .c ---
    // Redirect standard input from pipe's read end
    dup2(pipefd[0], STDIN_FILENO);

    // Close redundant pipe descriptors
    close(pipefd[0]);
    close(pipefd[1]);

    // Execute "grep .c"
    char *args[] = {"grep", ".c", NULL};
    execvp(args[0], args);

    perror("execvp grep failed");
    exit(EXIT_FAILURE);
}

// --- PARENT PROCESS ---
// Close parent's copies of pipe descriptors so readers/writers see EOF properly
close(pipefd[0]);
close(pipefd[1]);

// Wait for both child processes to exit
waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);

return 0;

```

```

bhanu@DESKTOP-DE32HQ8:~$ nano shell_pipe.c
bhanu@DESKTOP-DE32HQ8:~$ gcc shell_pipe.c -o shell_pipe
./shell_pipe
drwxr-xr-x 2 bhanu bhanu 4096 Aug  8 03:39 Projects
-rwxr-xr-x 1 bhanu bhanu 16336 Aug 22 03:32 copy_demo
-rw-r--r-- 1 bhanu bhanu 1461 Aug 22 03:24 copy_demo.c
-rw-r--r-- 1 bhanu bhanu 2589 Aug 11 06:31 main.c
-rwxr-xr-x 1 bhanu bhanu 16392 Aug 11 06:31 my_cli
-rwxr-xr-x 1 bhanu bhanu 16360 Aug 22 03:38 process_states
-rw-r--r-- 1 bhanu bhanu 1854 Aug 22 03:47 process_states.c
-rwxr-xr-x 1 bhanu bhanu 16488 Aug 22 03:18 prod_cons
-rwxr-xr-x 1 bhanu bhanu 16488 Aug 22 04:05 producer_consumer
-rw-r--r-- 1 bhanu bhanu 2243 Aug 22 03:19 producer_consumer.b
-rw-r--r-- 1 bhanu bhanu 2240 Aug 22 04:05 producer_consumer.c
-rw-r--r-- 1 bhanu bhanu 1999 Aug 22 03:12 shell_demo.c
-rw-r--r-- 1 bhanu bhanu 1679 Aug 22 04:07 shell_pipe.c
-rw-r--r-- 1 bhanu bhanu 1478 Aug 22 03:53 wait_comparison.c
-rw-r--r-- 1 bhanu bhanu 399 Aug  7 09:16 zombie.c
-rw-r--r-- 1 bhanu bhanu 613 Aug 22 03:57 zombie_demo.c
-rw-r--r-- 1 bhanu bhanu 1901 Aug 22 03:58 zombie_fix.c
bhanu@DESKTOP-DE32HQ8:~$ |

```